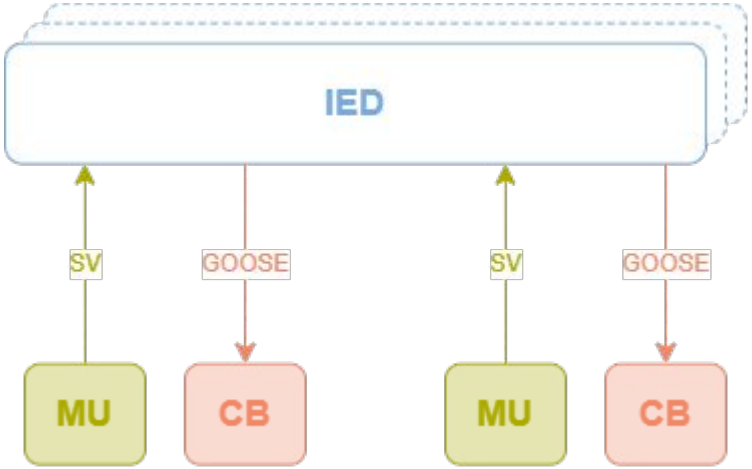
Scheduling for Deterministic Virtualization of Virtualized Protection and Control Workloads on COTS Hardware

Leon Simoniants
Advisors: Peter Okelmann, Dr. Anatole Lefort
Chair of Computer Systems
https://dse.in.tum.de/



20.05.2026 – 21.09.2026

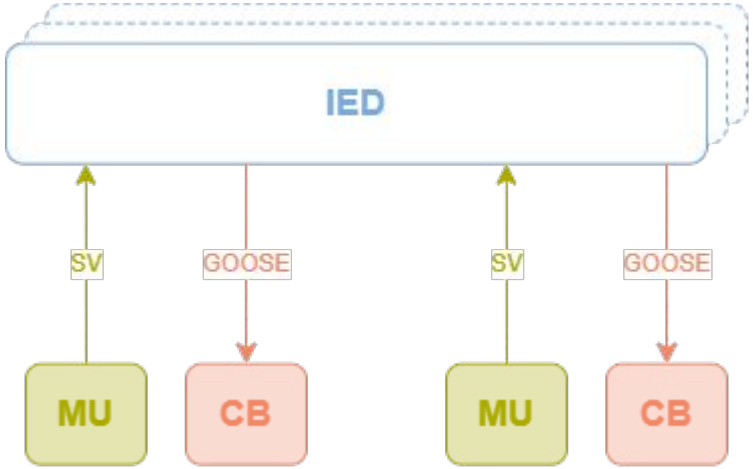
Motivation: Why virtualize protection? – Classical IEDs

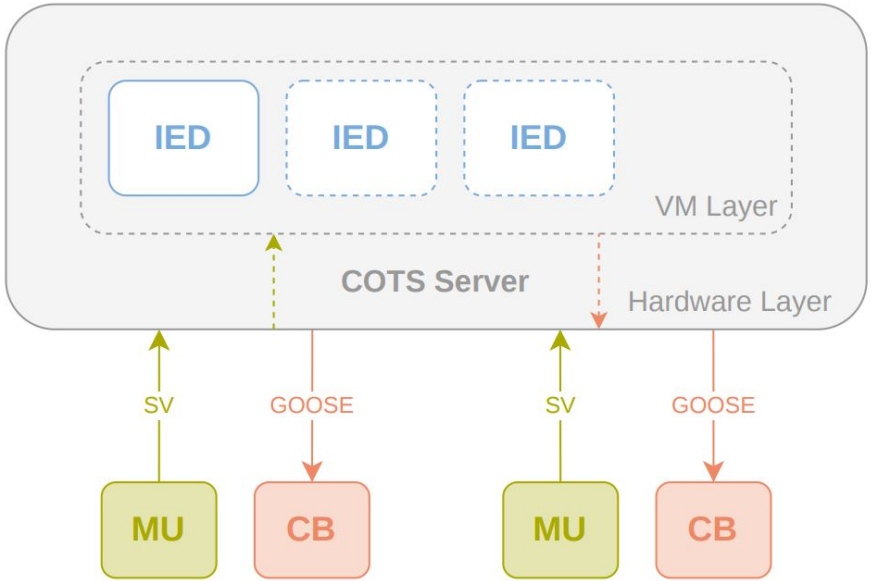


Classical IEDs

- many expensive, proprietary IEDs
- 5ms deadline trip

Motivation: Why virtualize protection? – Virtualization

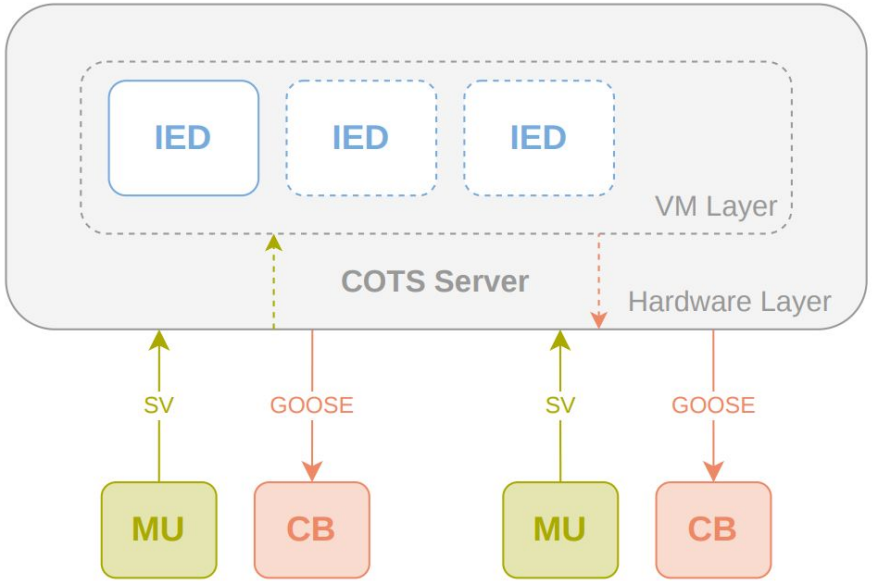




Motivation: Why virtualize protection? – vIEDs

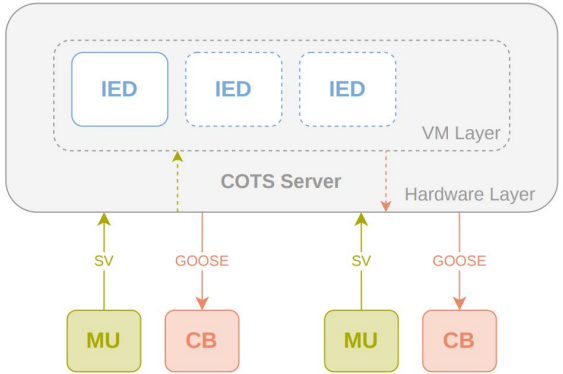
Virtualized IEDs (vIEDs)

- COTS hardware
- many vIEDs consolidated



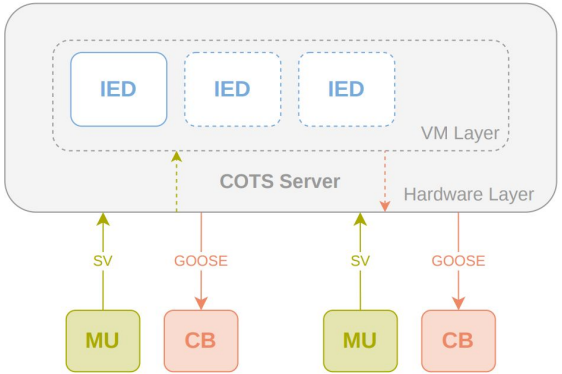
Research Gap: Synthesis vPAC and COTS RT-Linux (1)

- Virtualization on COTS brings
  - Scheduling
  - Interrupts
  - Housekeeping
  - …



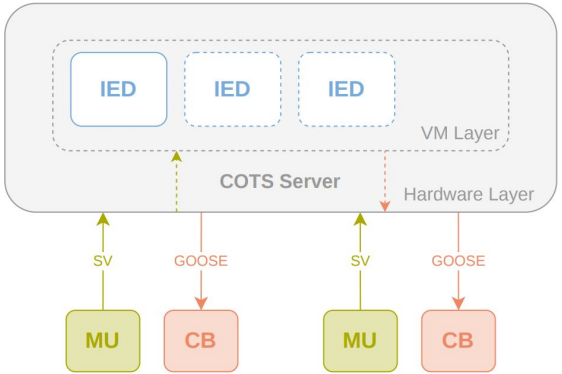
Research Gap: Synthesis vPAC and COTS RT-Linux (1)

- Virtualization on COTS brings
  - Scheduling
  - Interrupts
  - Housekeeping
  - …
- vPAC literature
  - system/application level



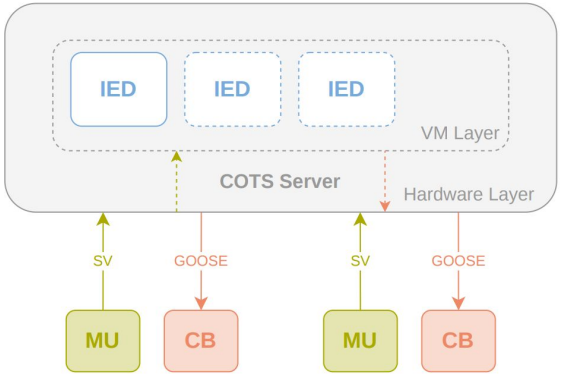
Research Gap: Synthesis vPAC and COTS RT-Linux (1)

- Virtualization on COTS brings
  - Scheduling
  - Interrupts
  - Housekeeping
  - …
- vPAC literature
  - system/application level
- RT-Linux literature
  - generic benchmarks



Research Gap: Synthesis vPAC and COTS RT-Linux (1)

- Virtualization on COTS brings
  - Scheduling
  - Interrupts
  - Housekeeping
  - …



- vPAC literature
  - system/application level

→ **vPAC feasibility**

- RT-Linux literature
  - generic benchmarks

→ **RT-Linux tuning**

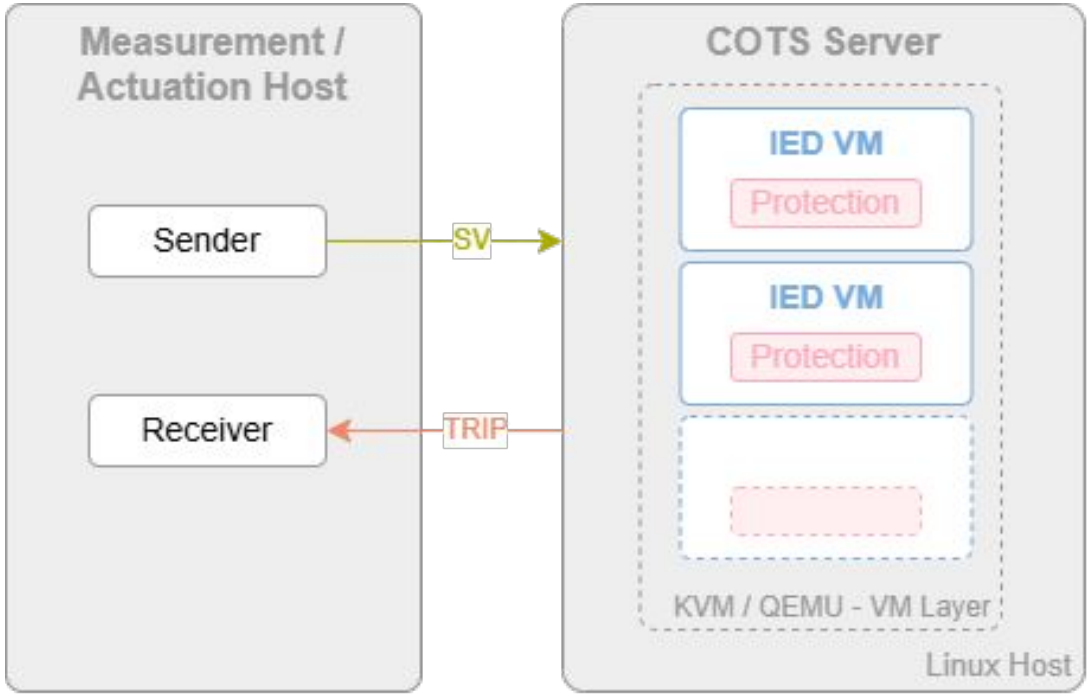
Research Gap: Synthesis vPAC and COTS RT-Linux (2)

vPAC feasibility + RT-Linux tuning

- concrete scheduling and virtualization mechanism behavior for vPAC timings

- COTS hardware + Linux (PREEMPT_RT)

System and representative workload – Physical Hosts



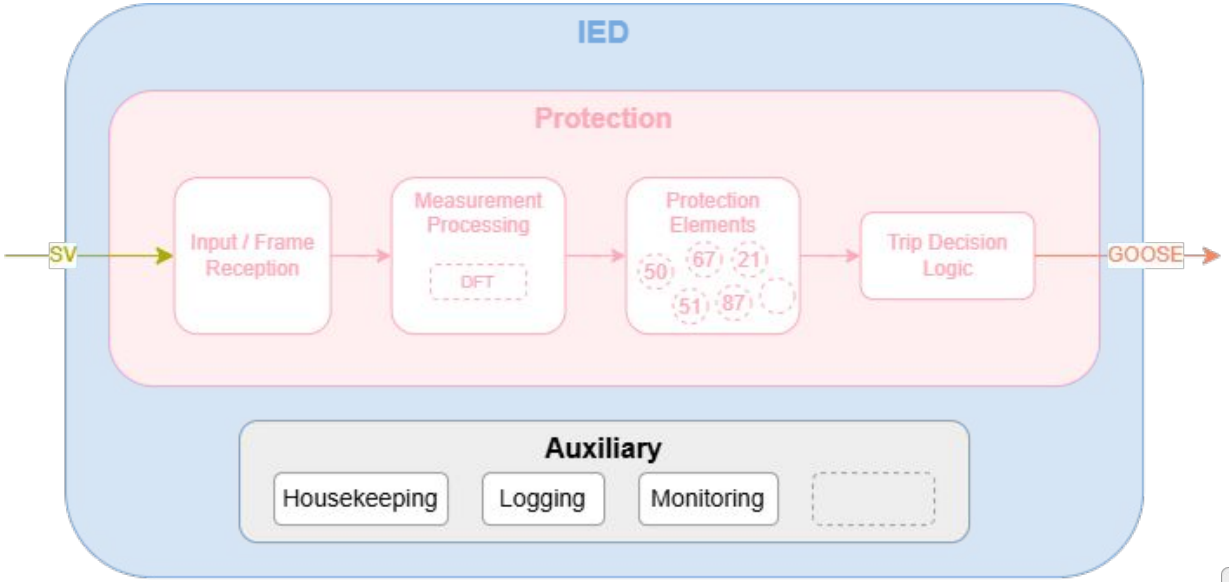
Two physical hosts

5ms trip time:

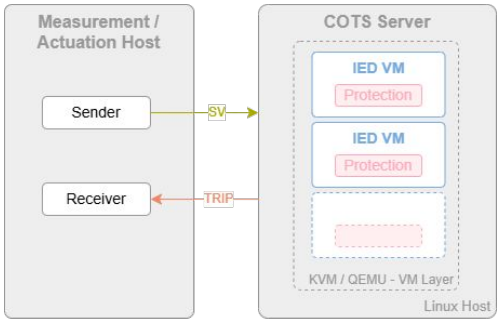
– sender - prot. - recv.

vIEDs as VMs

System and representative workload – vIED VM



- 6 open-source implementations
- protection:
  ~ 1.19µs / invocation



Scheduling design – General

- tested low-latency kernel parameters and configuration

- **Scheduling Priority Hierarchy**
  - [98] hot-path network IRQs (Rx/Tx)
  - [95] protection process
  - [>50] auxiliary RT

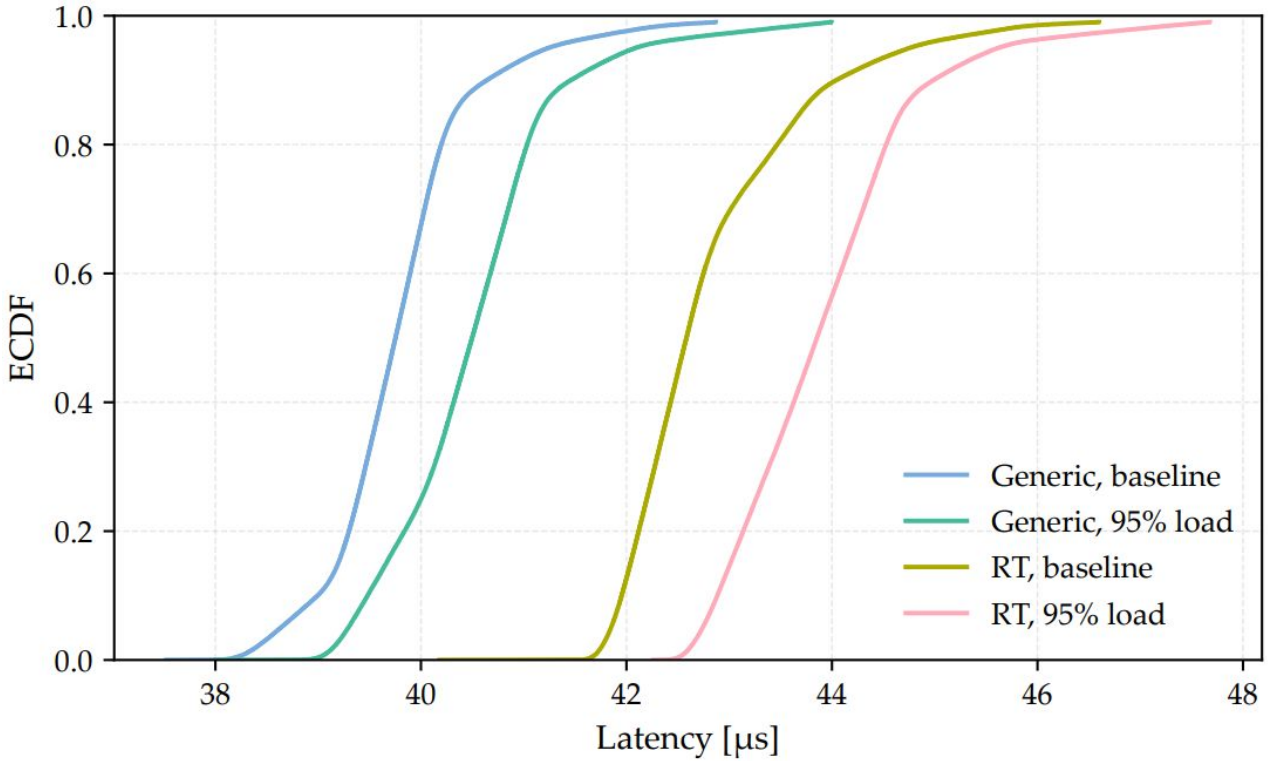
  - ([50] default IRQ threads)
  - (ordinary / fair-scheduled work)

Scheduling design – VM Topologies

- Two VM-topology designs: **1-vCPU** & **2-vCPU**

1-vCPU	**2-vCPU**
All work **must** share one CPU0	Housekeeping (CPU0) and RT (CPU1) **separate**
Requires strict prio. and workload control	Yield stronger isolation and fault containment between workloads
Less backing physical resources	**More** backing physical resources
Goal: resource efficiency	**Goal: isolation & robustness**

Finding 1: PREEMPT_RT limitations – General RTT



three runs per ECDF

100,000 packets per run

\+ 95% CPU load

Both kernels perform stable under latency tuning in general case

Finding 1: PREEMPT_RT limitations – Latency Tails

Topology	Kernel	Mean	p99	p99.9	p99.99	Maxima
1 vCPU	Generic	3.275	5	6	11	18-27
	PREEMPT_RT	3.179	4	5	7	8-11
2 vCPU (both placements)	Generic	2.075	3	4	7	21-35
	PREEMPT_RT	2.222	4	5	9	10-14

Guest-Visible Scheduling Latency [µs] - via cyclictest

\+ Additional **sync Disk-I/O** load

(3 x 120s runs - 120,000 wake-ups per run)

⇒ We see difference between kernels → PREEMPT_RT **tighter tail**

Finding 1: PREEMPT_RT limitations – Result

- PREEMPT_RT is **not** a low-latency switch

- simply enabling, does **not** make system have “lower latency”

However, it **increases kernel preemptability** and yields **control** (e.g. threaded IRQs)

Finding 2: Priority Hierarchy

Condition	Received	Median	p99.9	> 5ms
PREEMPT_RT, RT-load 90, **netIRQ 50**	~84 %	~90 ms	~400 ms	~95 %
PREEMPT_RT, RT-load 90, **netIRQ 98**	100 %	41.6 µs	~48 µs	0 %

- Extra 95% RT CPU-load with priority 90

- netIRQ 50 -> packet loss & inacceptable latency

If utilizing PREEMPT_RT, a **use-case appropriate priority hierarchy** must be **considered carefully**

Finding 3: Workload Separation – Default

Problem:

RT tasks may **starve** system

- Consume entire CPU

Fair-Server:

If non-RT runnable

→ provide execution time

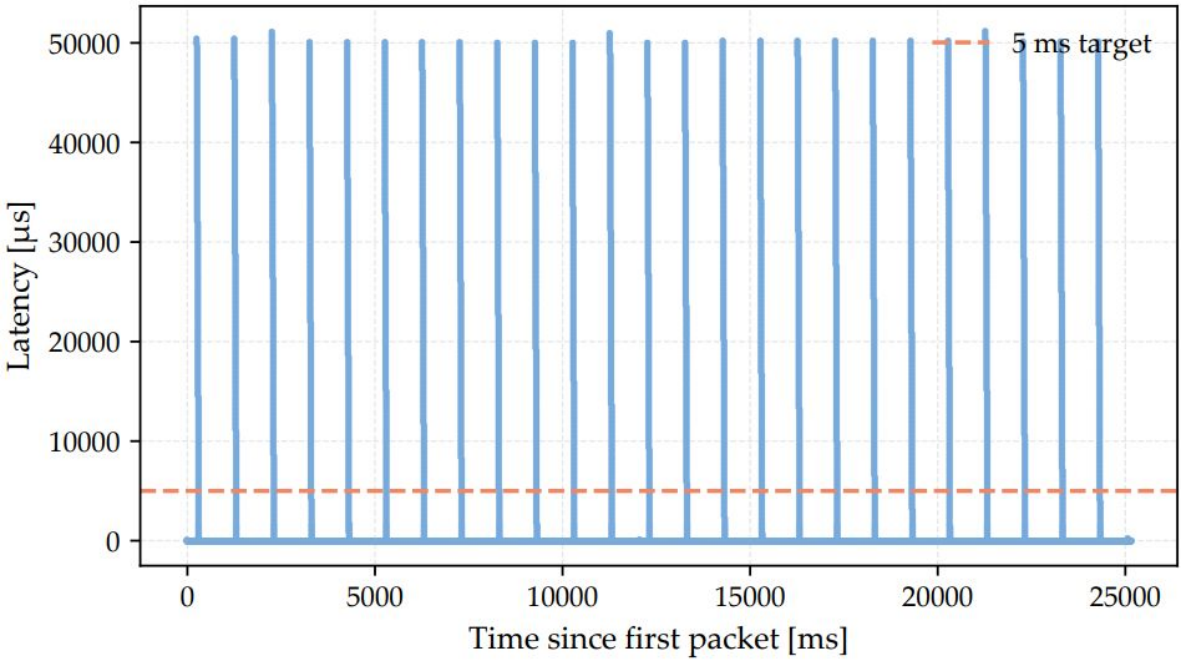
Default: 50 ms / 1 s

Finding 3: Workload Separation – Problem

fair-server enabled

+ RT 95% CPU-load

+ non-RT runnable process



Fair-server effects unacceptable for latency requirements.

Finding 3: Workload Separation – Result

Generally, proper workload admission is critical.

⇒ **2-vCPU** separates RT and non-RT via vCPU pinning intentionally

→ Can keep fair-server enabled

⇒ **1-vCPU** inherently needs to co-locate RT and non-RT

→ Disable fair-server

→ **Requires strict RT workload admission control**

Additional Findings

- **VM-exit** timing investigation under different load and causes

  Exit path matters – exit frequency not so much

  Avoid legacy userspace-emulated I/O – keep minimal VM device model

- **1vCPU vs. 2vCPU**

  Similar latency ; different trade-off – none inherently superior

- **Multi-VM** (i.e. multiple vIEDs)

  Tested up to 7-8 concurrent VMs across both topologies

Does our system work? – Long-running measurement

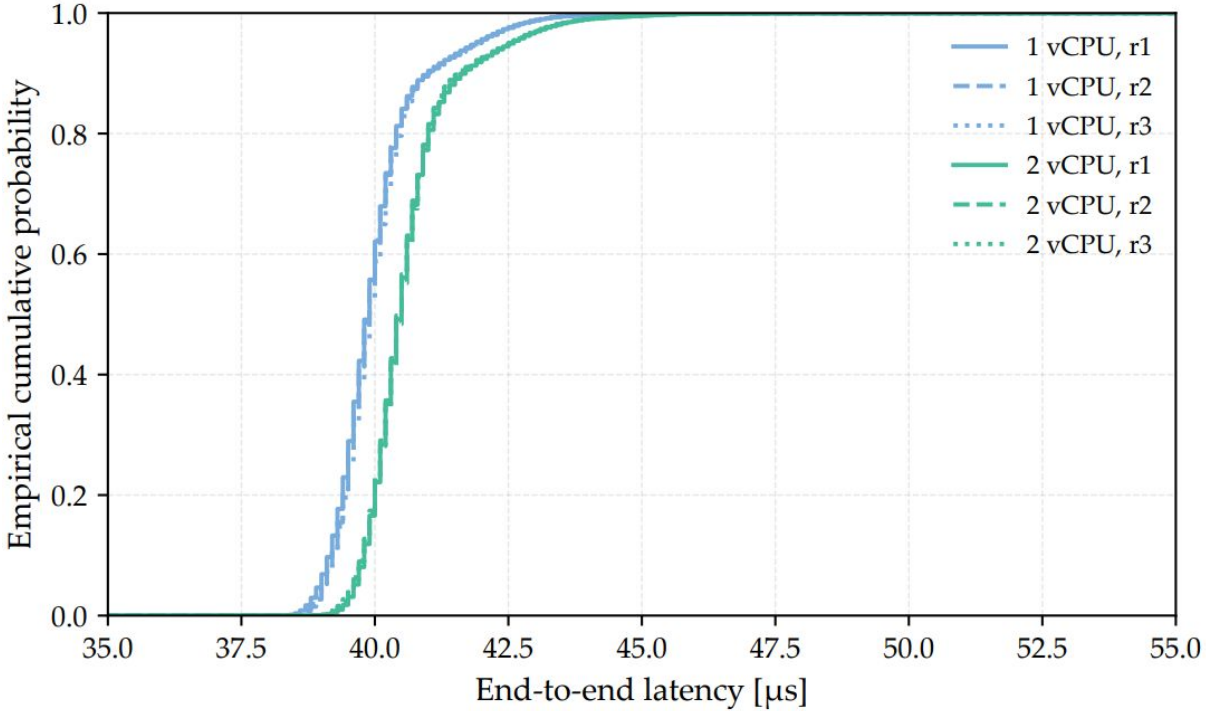
First of all; host-side network optimization and delivery is not in scope (Radu)

Apart from that:

- 3 x 1h runs per topology:
  - 3 runs x 14.4 million frames x 2 topologies ⇒ **86.4 million** end-to-end samples
- Dense distributions around **40 µs – 46 µs** — very reproducible
- Typically; no latencies exceed 1 ms

- However, we did record a rare transient event (for **both** topologies once)

Does our system work? – Dense ECDF

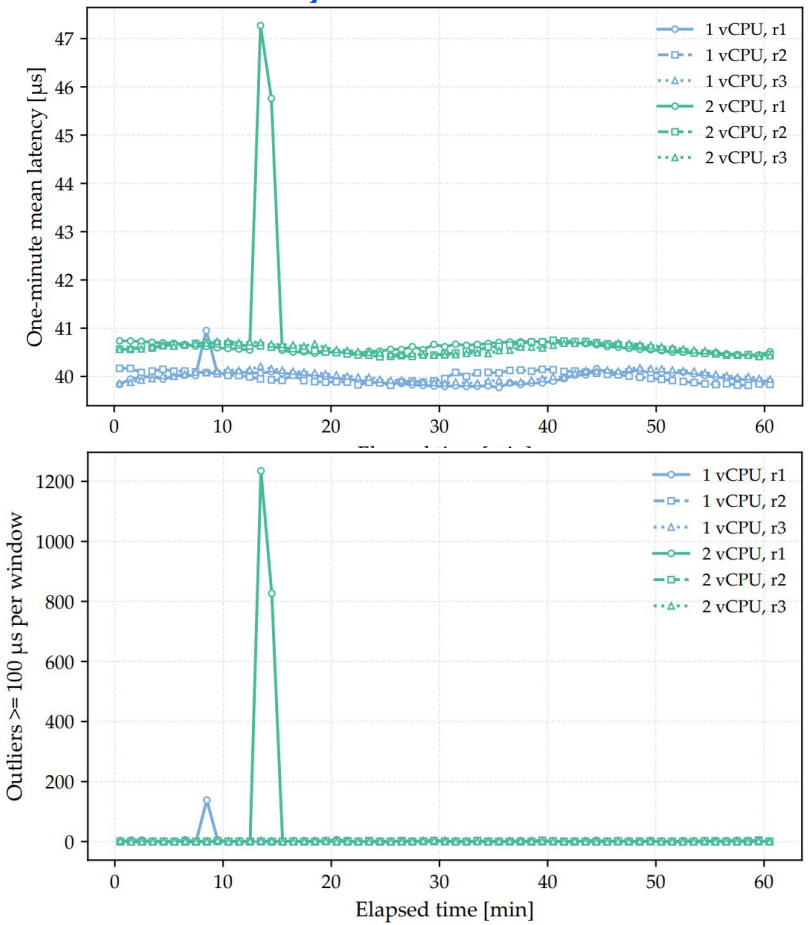


1vCPU – blue

2vCPU – green

All runs per topology very similar → reproducible latency

Does our system work? – Rare Transient Event



- Per-Minute Windows
  - Mean and “Outliers” (>= 100 µs)
- Excluding rare-event, runs typically:
  - 33-65 >= 100 µs / run (14.4 mil)
  - **No** latency > 1 ms
- No system degradation over time
- Maxima of 3 ms and 4.3 ms
  - During rare event
- Within events, frame latencies decrease by ~250 µs

Conclusion

⇒ Low and reproducible latency is feasible.

⇒ Scheduling/isolation configuration matters; not simply enabling PREEMPT_RT.

⇒ Remaining E2E tails require more detailed path-level localization.

⇒ It would be interesting to combine this and the host networking work and potentially instrument that system to attribute remaining tail latency

Questions?